Esercizio S3-L4

1) Messaggio cifrato: “HSNFRGH”

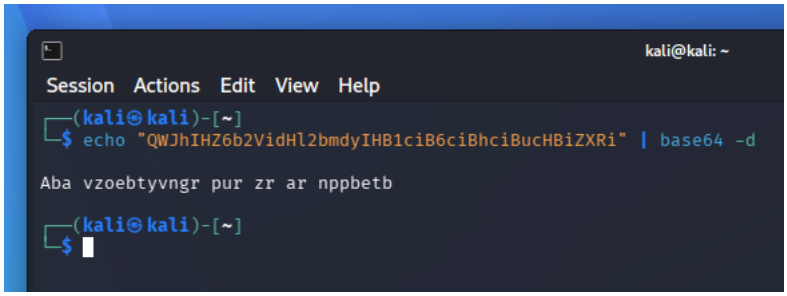
Per decrittografare questo messaggio bisogna utilizzare il metodo del Cifrario di Cesare.
In questo particolare caso ogni lettera deve essere spostata di 3 posizioni indietro, ma considerando l’alfabeto italiano.

H -> E
S -> P
N -> I
F -> C
R -> O
G -> D
H -> E

Il messaggio decriptato è “EPICODE”

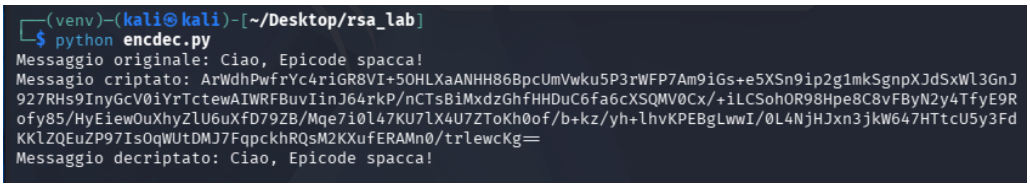
2) Messaggio cifrato: “QWJhIHZ6b2VidHl2bmdyIHB1ciB6ciBhciBucHBiZXRi”

Per decifrare questo messaggio è necessario prima inserirlo su kali e decriptarlo con | base64 -d



A questo punto, come sopra, è necessario utilizzare il Cifrario di Cesare.
In questo caso specifico le lettere vanno spostate di 13 in avanti, ma dell’alfabeto inglese.

3)



```
(venv)kali@kali: ~/Desktop/rsa_lab
Session  Actions  Edit  View  Help
  GNU nano 8.7.1                                    encdec.py
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import serialization
import base64

with open('private_key.pem', 'rb') as key_file:
        private_key = serialization.load_pem_private_key (
                key_file.read(),
                password=None)

with open('public_key.pem', 'rb') as key_file:
        public_key = serialization.load_pem_public_key(key_file.read())

message = 'Ciao, Epicode spacca!'

encrypted = public_key.encrypt(message.encode(), padding.PKCS1v15())

decrypted = private_key.decrypt( encrypted, padding.PKCS1v15())

print("Messaggio originale:", message)
print("Messagio criptato:", base64.b64encode(encrypted).decode('utf-8'))
print("Messaggio decriptato:", decrypted.decode('utf-8'))

^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Loc
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go
```

4)

```
┌──(venv)─(kali㉿kali)-[~/Desktop/rsa_lab]
└─$ nano firma.py

┌──(venv)─(kali㉿kali)-[~/Desktop/rsa_lab]
└─$ python firma.py
Base64 della firma: vWxAhpgAbTFljexMPZWnhYc3ZPJ1dpO7XfPvCSLkj0dsuL2pOWCiqo+kSQlYMv1XJqxOsdGG6R/49T1NRwy
MqnBf1kVhMf0mYoFphgLxv5UnfOnfgK5Dr2lHRvr0mDOgXvI4917tXLid3zKENl76vMI5wUds+yIGh8F3excEGCQO9cC46+wtKAdvHx
+Y9H5HFhQ8Qi+n1gMw7BwDSEbHA/RZdLubRYEj6c1S6MQDx06eId12B7+xd4W6Gcb7zksQWRB8Hz950HM3eDIEPyCWp5MntV5bLvoc5
TxjcLmnXpPK4T6Vt4JIPUGmoaal1dspPa/qfUYFkmTmoZROtVvyCA==
Messaggio originale da confrontare: Ciao, Epicode spacca!
La firma è valida.

┌──(venv)─(kali㉿kali)-[~/Desktop/rsa_lab]
└─$
```

```
(venv)kali@kali: ~/Desktop/rsa_lab
Session  Actions  Edit  View  Help
  GNU nano 8.7.1                                    firma.py
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives import serialization
import base64

with open('private_key.pem', 'rb') as key_file:
    private_key = serialization.load_pem_private_key(
        key_file.read(),
        password=None)

with open('public_key.pem', 'rb') as key_file:
    public_key = serialization.load_pem_public_key(
        key_file.read()
    )

message = 'Ciao, Epicode spacca!'

signed = private_key.sign(
    message.encode(),
    padding.PKCS1v15(),
    hashes.SHA256()
)

try:
    encrypted_b64 = base64.b64encode(signed).decode('utf-8')

    public_key.verify(
        signed,
        message.encode(),
        padding.PKCS1v15(),
        hashes.SHA256()
    )

    print("Base64 della firma:", encrypted_b64)
    print("Messaggio originale da confrontare:", message)
    print("La firma è valida.")

except Exception as e:
    print("La firma non è valida.", str(e))

^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```